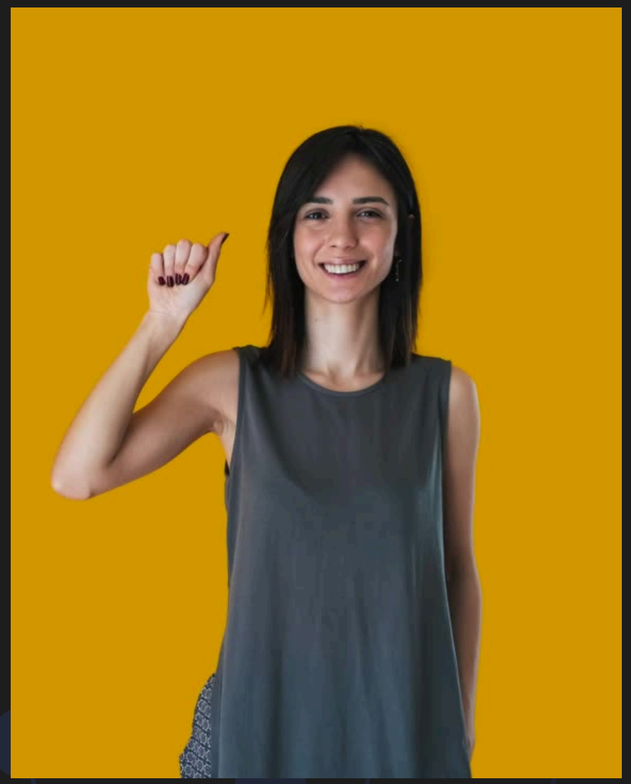
LSTM-Based Sign Language Recognition

Matthew Hema

What if?



Why not?

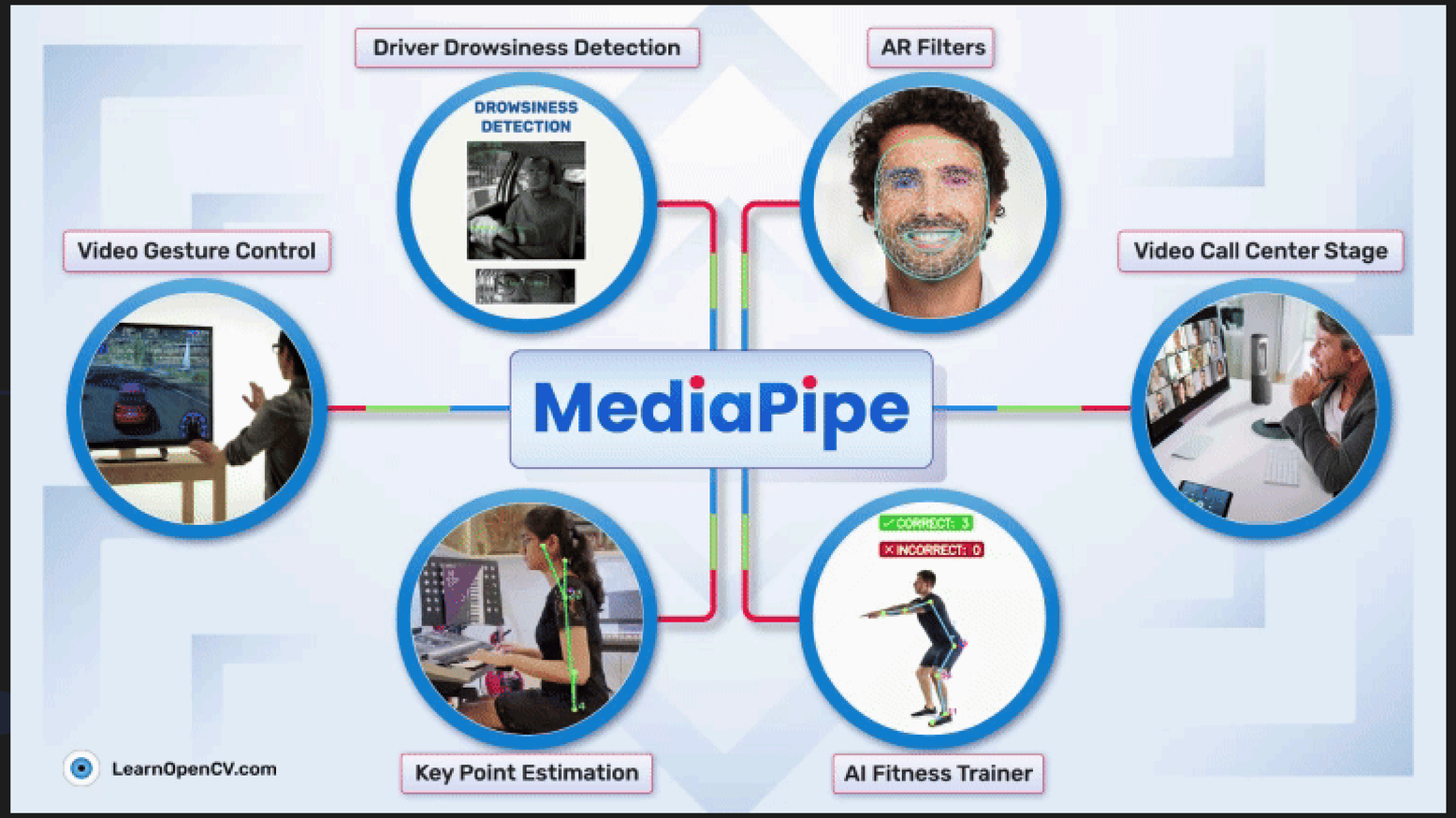


Intelligent
Smart
Efficient
Model

“Hello”
“Sorry”
“Hungry”
“Yes No”
“Go Home”

Mediapipe

MediaPipe is an open-source framework developed by Google that enables developers and researchers to build machine learning pipelines, especially for computer vision and multimedia applications.

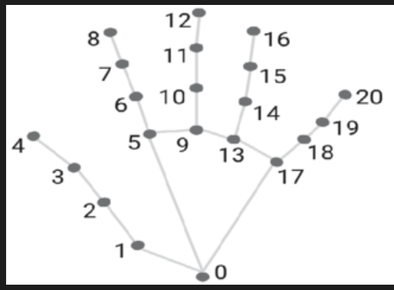


(Learnopencv.com)

Mediapipe

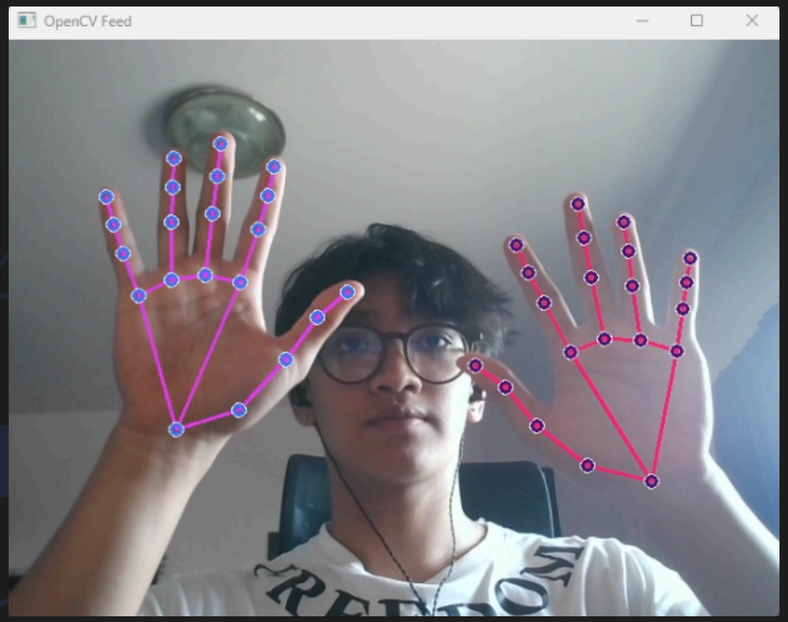
mp_holistic.HAND_CONNECTIONS

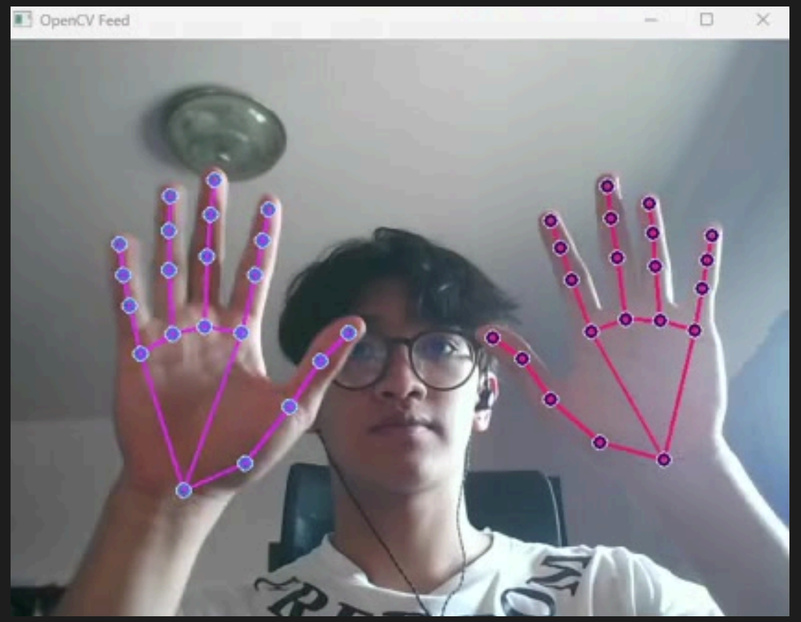
21 positions for each hand connection are collected.



0. WRIST
1. THUMB_CMC
2. THUMB_MCP
3. THUMB_IP
4. THUMB_TIP
5. INDEX_FINGER_MCP
6. INDEX_FINGER_PIP
7. INDEX_FINGER_DIP
8. INDEX_FINGER_TIP
9. MIDDLE_FINGER_MCP
10. MIDDLE_FINGER_PIP
11. MIDDLE_FINGER_DIP
12. MIDDLE_FINGER_TIP
13. RING_FINGER_MCP
14. RING_FINGER_PIP
15. RING_FINGER_DIP
16. RING_FINGER_TIP
17. PINKY_MCP
18. PINKY_PIP
19. PINKY_DIP
20. PINKY_TIP

(Divyansh Bisht,2022)





OpenCV

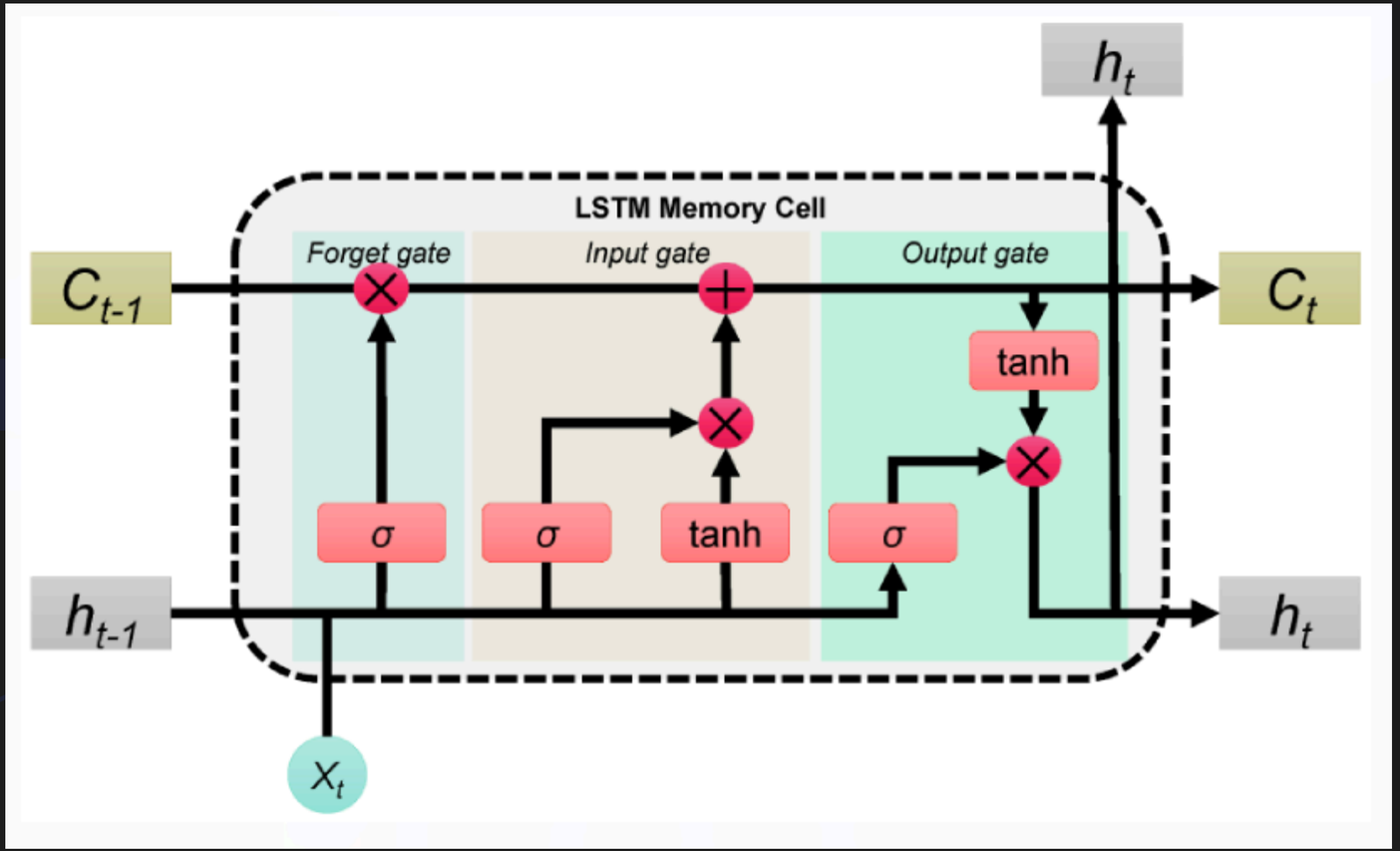
OpenCV (Open Source Computer Vision Library) is a widely used open-source library designed for computer vision and machine learning tasks. Initially developed by Intel.



(Roboflow)

LSTM

Long Short-Term Memory (LSTM) is a specialized type of recurrent neural network (RNN) designed to handle and learn from sequential data with long-term dependencies.



(MLArchive)

LSTM

My name is Somchai, it shouldn’t be hard to guess that I’m from ..... (Where?)

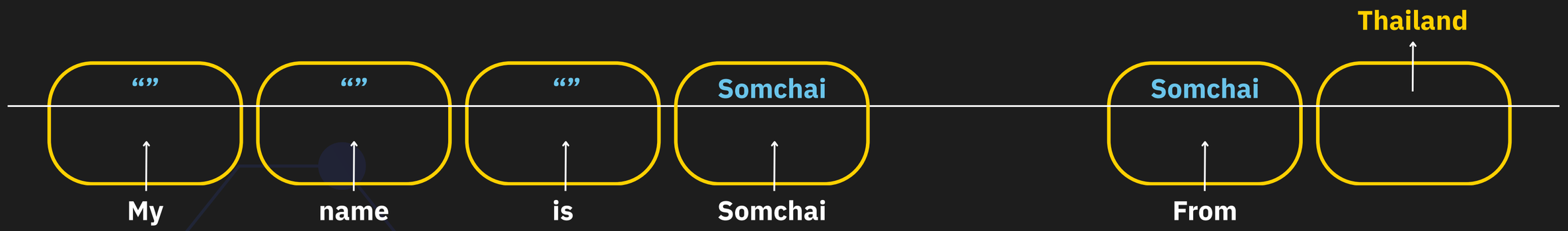
My name is Somchai, it shouldn’t be hard to guess that I’m from **Thailand**

My name is **Somchai**, it shouldn’t be hard to guess that I’m from **Thailand**

LSTM

RNN (Recurrent Neural Network)

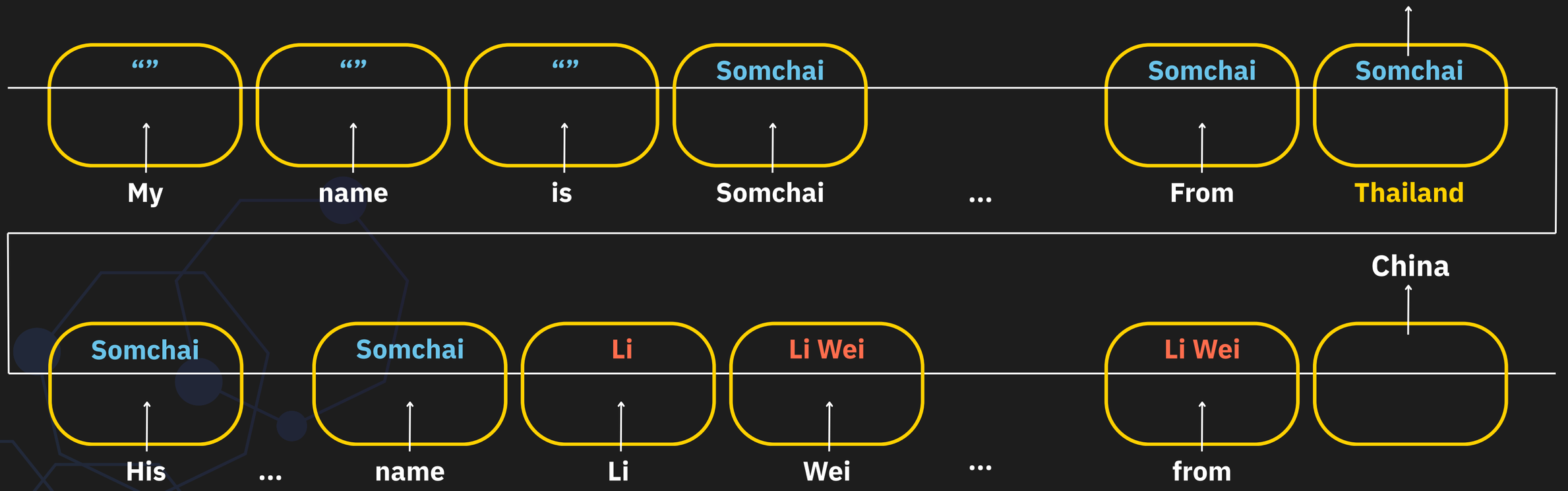
My name is Somchai, it shouldn’t be hard to guess that I’m from **Thailand**



LSTM

Long Short Term Memory

My name is Somchai, it shouldn’t be hard to guess that I’m from Thailand.
However, his friend’s name Li Wei, which suggests she is from China.



Prototype

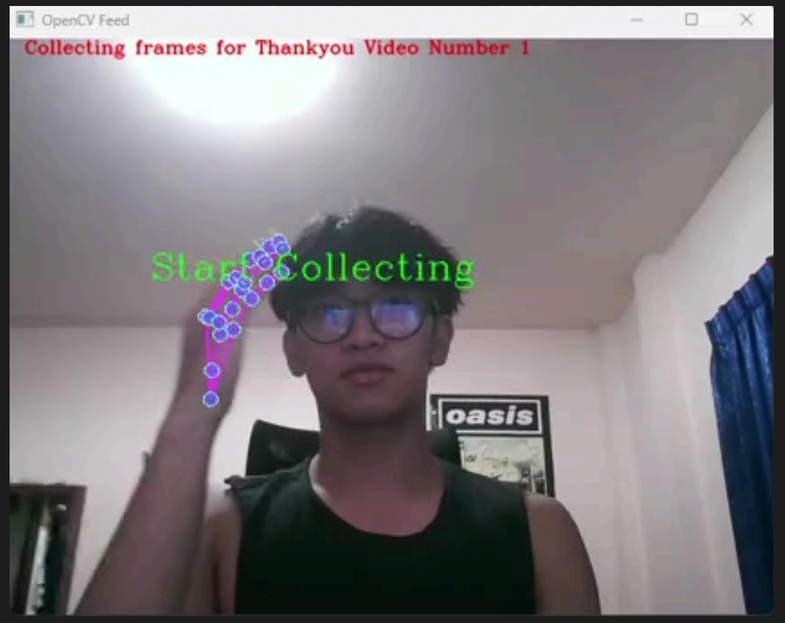
Data Collection

30 sequences + (about 2 seconds)
Collect each sign 30 times
40 cm distance from camera

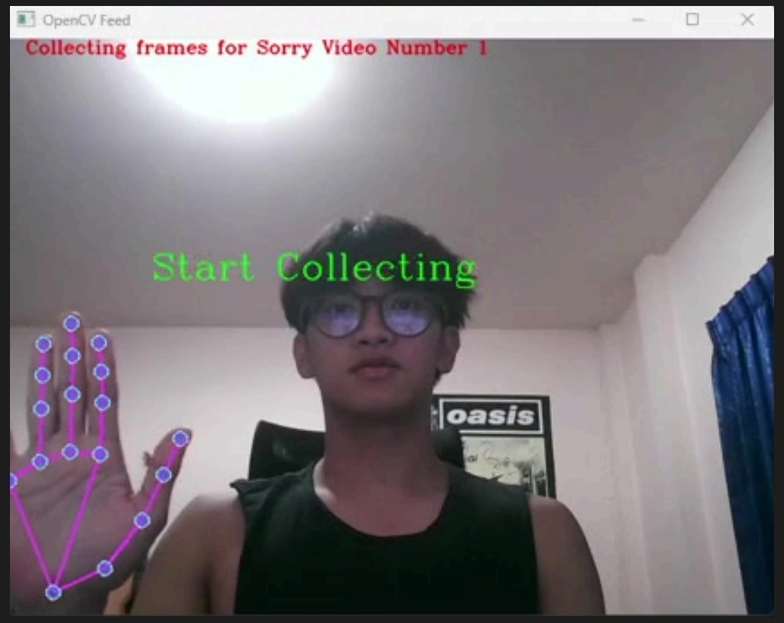
“Hello”



“Thank you”



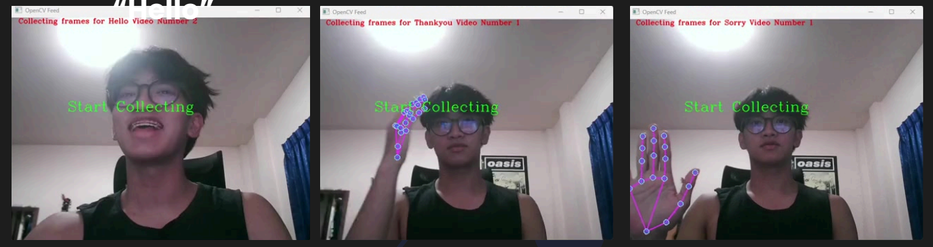
“Sorry”



Data Dimension

Sequences.shape = (90,30,126)

90



30 recorded each sign

= 30x3 = 90

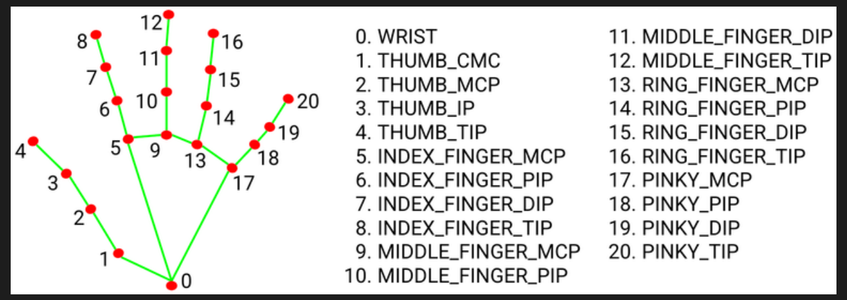
30

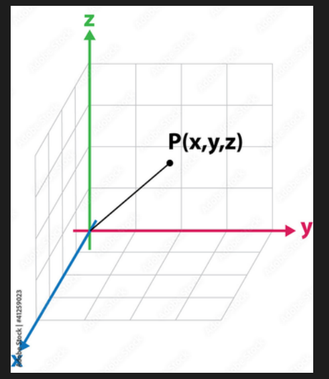
collected each record

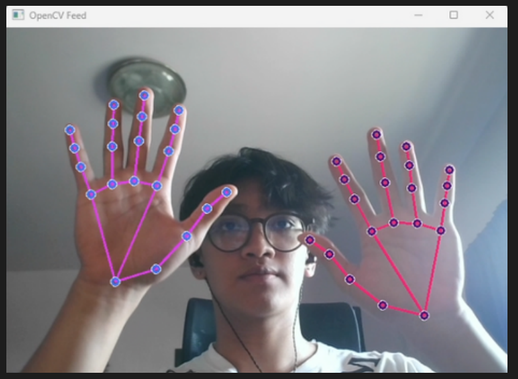
30 frames (sequences)

About 2 seconds

126







= 21 x 3 x 2 = 126

Train Test Split

Split Train Test: 95% and 5%

Train Dimension : 85, 30, 126

Test Dimension : 5, 30, 126

```python
model2 = Sequential()
model2.add(LSTM(64,return_sequences=True, activation='relu', input_shape=(30,126)))
model2.add(LSTM(128,return_sequences=True, activation='relu'))
model2.add(LSTM(64,return_sequences=False, activation='relu'))
model2.add(Dense(64, activation='relu'))
model2.add(Dense(32, activation='relu'))
model2.add(Dense(3, activation='softmax'))
```

Python

```python
model.compile(optimizer='Adam', loss='categorical_crossentropy', metrics=['categorical_accuracy'])
```

Python

```python
model.fit(X_train,y_train, epochs=2000, callbacks=[tb_callback])
```

Python

Model

Long Short Term Memory

```
Model: "sequential_1"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 lstm_3 (LSTM)               (None, 30, 64)            48896

 lstm_4 (LSTM)               (None, 30, 128)           98816

 lstm_5 (LSTM)               (None, 64)                49408

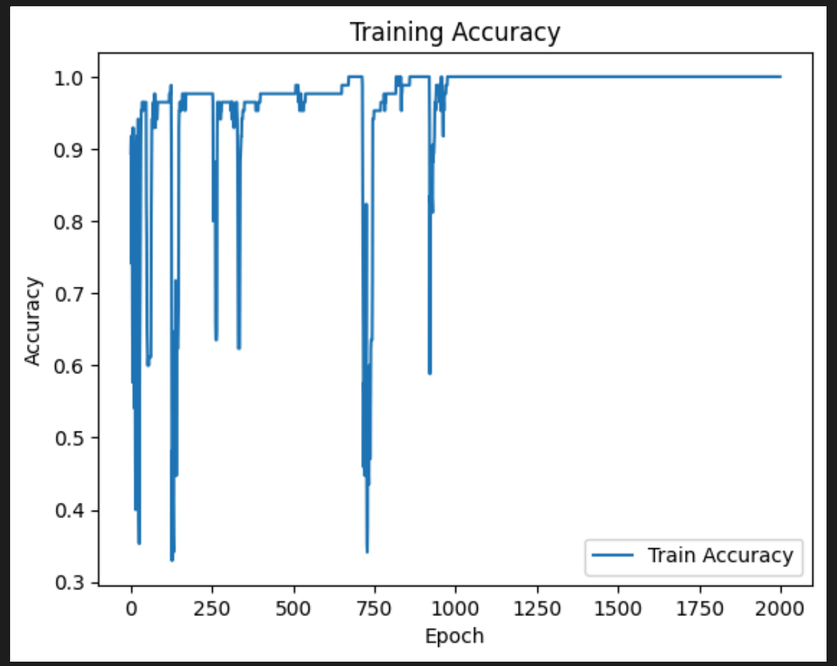
 dense_3 (Dense)             (None, 64)                4160

 dense_4 (Dense)             (None, 32)                2080

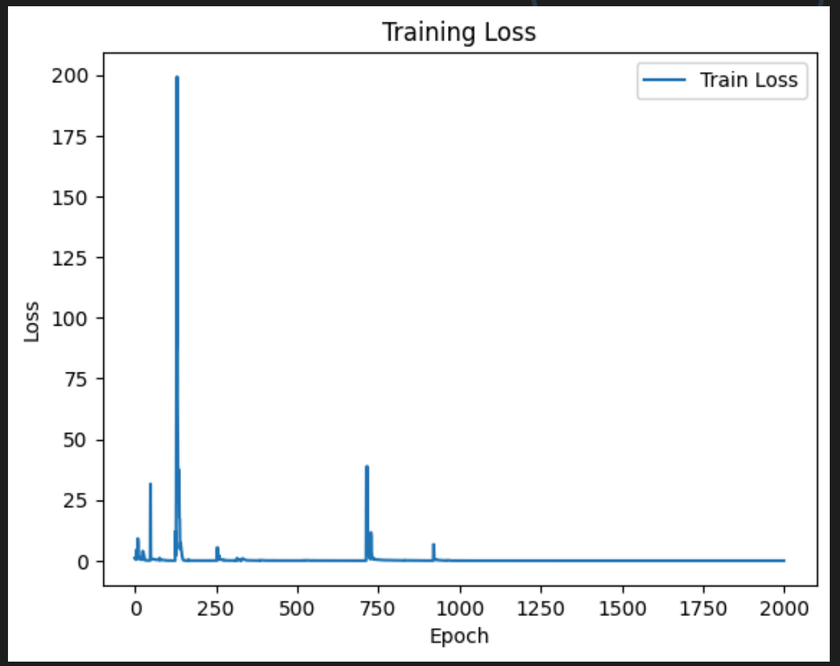
 dense_5 (Dense)             (None, 3)                 99

=================================================================
Total params: 203459 (794.76 KB)
Trainable params: 203459 (794.76 KB)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
```

Model Summary



Final training loss : 2.284598622e-06

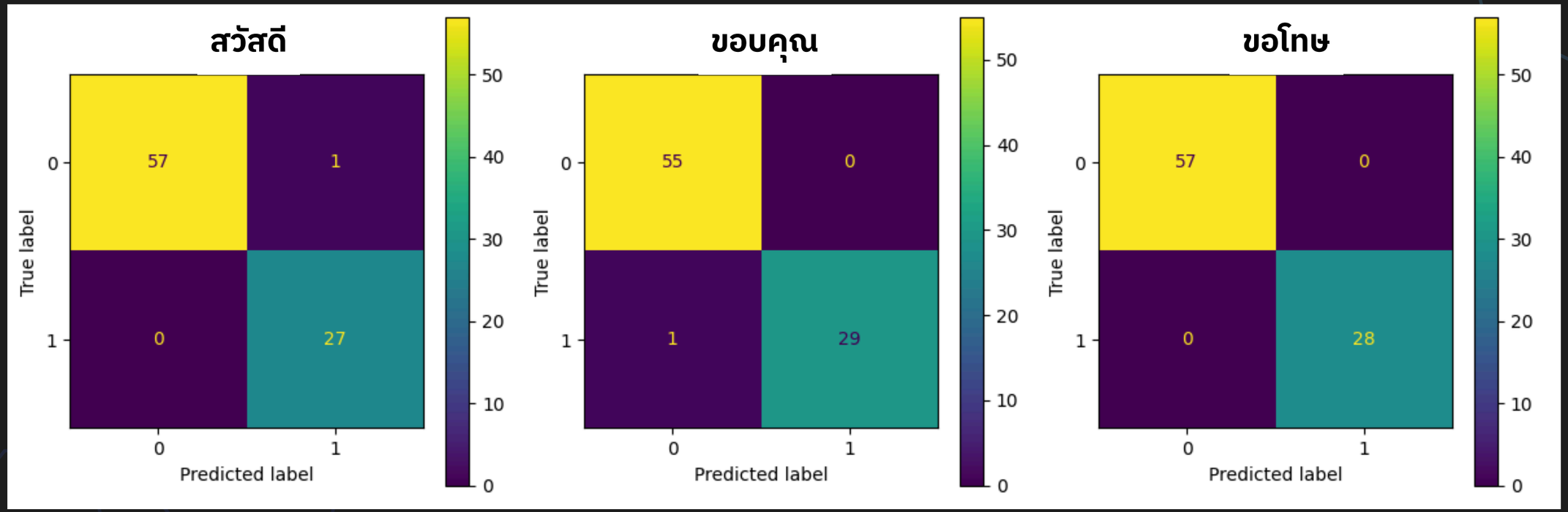


Final training accuracy : 1.0

2000 epochs

Model

Long Short Term Memory



Accuracy Score : 0.9882353

Voice Generator

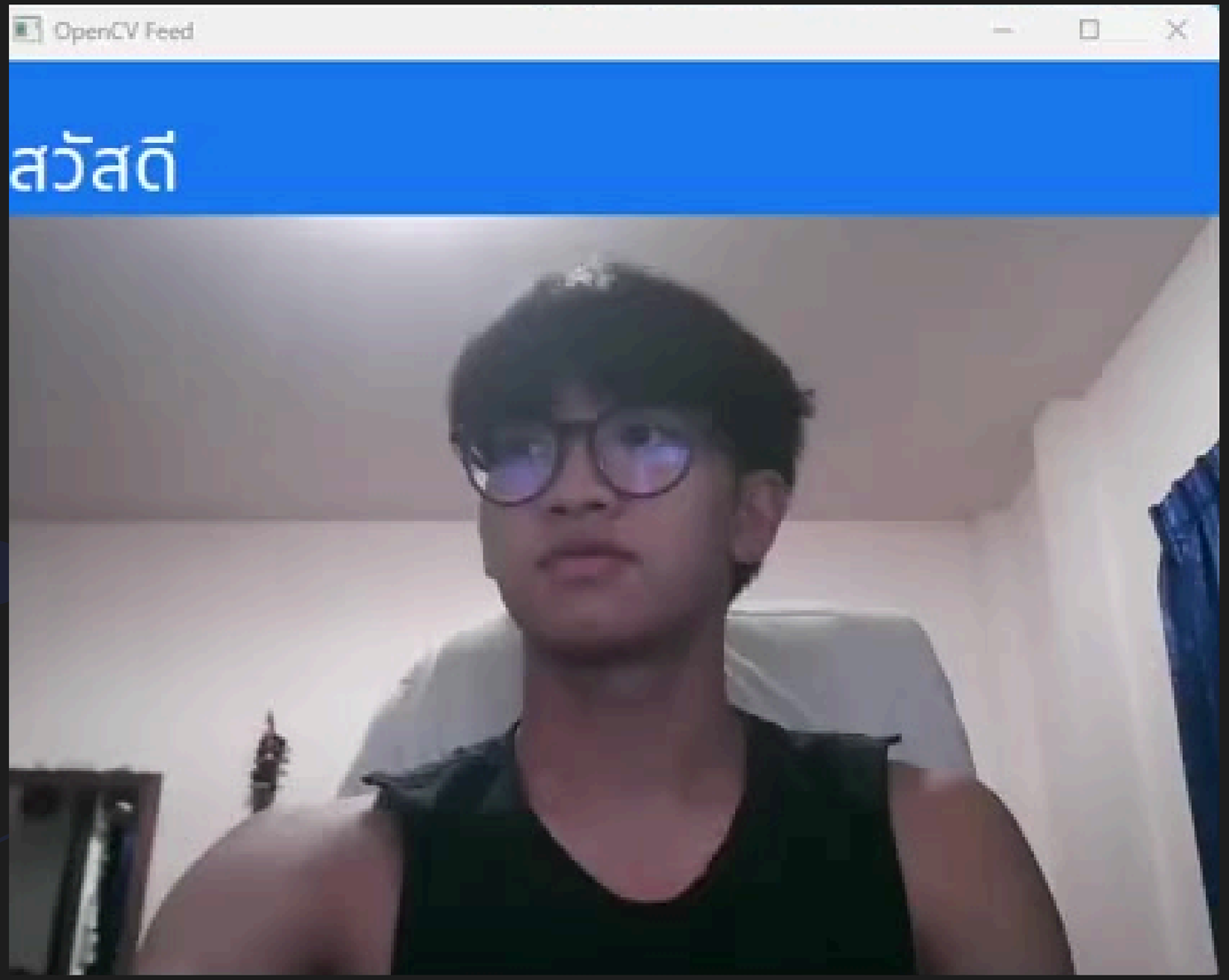
gtts (Google Text-to-Speech)

```python
from gtts import gTTS
import playsound
txs = gTTS(text='สวัสดี', lang='th')
txs.save('01.mp3')
playsound.playsound('01.mp3')
```

example code



Demonstration



Main Function

```python
def mediapipe_detection(image, model):
    image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    image.flags.writeable = False
    results = model.process(image)
    image.flags.writeable = True
    image = cv2.cvtColor(image, cv2.COLOR_RGB2BGR)
    return image, results
```

✓ 0.0s Python

```python
def draw_landmarks(image, results):
    mp_drawing.draw_landmarks(image,results.left_hand_landmarks, mp_holistic.HAND_CONNECTIONS)
    mp_drawing.draw_landmarks(image,results.right_hand_landmarks, mp_holistic.HAND_CONNECTIONS)
```

✓ 0.0s Python

```python
def draw_styled_landmarks(image, results):
    mp_drawing.draw_landmarks(image,results.left_hand_landmarks, mp_holistic.HAND_CONNECTIONS,
                              mp_drawing.DrawingSpec(color=(121,22,76), thickness=2,circle_radius=4),
                              mp_drawing.DrawingSpec(color=(121,44,250), thickness=2,circle_radius=2)
                              )

    mp_drawing.draw_landmarks(image,results.right_hand_landmarks, mp_holistic.HAND_CONNECTIONS,
                              mp_drawing.DrawingSpec(color=(245,117,66), thickness=2,circle_radius=4),
                              mp_drawing.DrawingSpec(color=(245,66,230), thickness=2,circle_radius=2)
                             )
```

✓ 0.0s Python

```python
def extract_keypoints(results):
    righthand = (np.array([[res.x, res.y, res.z] for res in results.right_hand_landmarks.landmark]).flatten()
                 if results.right_hand_landmarks else np.zeros(21*3))

    lefthand = (np.array([[res.x, res.y, res.z] for res in results.left_hand_landmarks.landmark]).flatten()
                if results.left_hand_landmarks else np.zeros(21*3))

    return np.concatenate([lefthand, righthand])
```

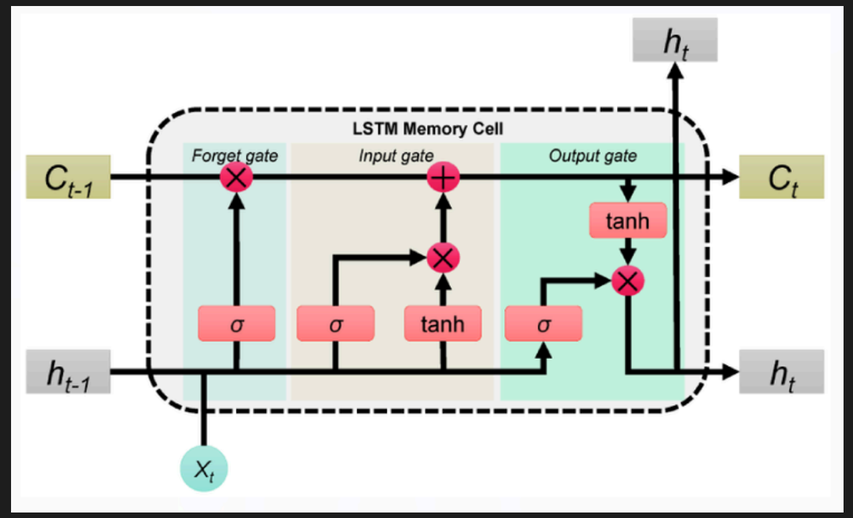
✓ 0.0s Python

Future Plan

- Expand the dataset by collecting additional words
- Optimize the model for greater efficiency
- Deploy the script on a website or mobile app
- Incorporate additional features—beyond hand connectors—in the data collection process



30 additional words



Tune + optimize LSTM



Deploy Project